TP : Développement d'une GED (Gestion Électronique de Documents) en PHP

Contexte

La société **OrgaPlus** souhaite développer l'application **MediaBox** afin de gérer à la fois des documents (pdf, images, sons, vidéos) et des liens web sous la forme de favoris.

OrgaPlus, au service de clients dans divers secteurs d'activité, vous laisse sélectionner des contenus selon vos préférences, afin d'expérimenter l'application et proposer un prototype ergonomique et attrayant.

Terminologie : dans ce TP, le terme **ressource** désigne indifféremment un *document* (fichier téléversé, de type « document ») ou un *favori* (lien vers une page web, de type « favori »). Les deux sont décrits par un enregistrement dans data/ressources.txt et sont affichés dans la même liste.

Quelques pistes d'utilisation de l'application selon différents domaines :

- **Jeux vidéos**
  Contexte : Organiser des guides, astuces, builds et liens vers des vidéos YouTube.
  Documents / Liens : PDF, images, captures d'écran, vidéos en ligne.
- **Cinéma / école de cinéma**
  Contexte : Stocker critiques d'élèves, bandes-annonces YouTube, affiches uploadées.
  Documents / Liens : PDF, images, vidéos en ligne.
- **Musique / club musical**
  Contexte : Gérer analyses de morceaux, liens YouTube, pochettes d'albums.
  Documents / Liens : PDF, images, liens audio.
- **Voyages / carnet collaboratif**
  Contexte : Stocker fiches de voyage (textes et photos), liens Google Maps ou blogs.
  Documents / Liens : PDF, images, liens web.
- **Études / gestion de fiches de révision**
  Contexte : Centraliser fiches de révision, liens vers cours en ligne, résumés étudiants.
  Documents / Liens : PDF, images, liens web.
- **Art / galerie numérique**
  Contexte : Illustrations uploadées, fiches d'œuvre, liens vers portfolios (Behance, ArtStation).
  Documents / Liens : PDF, images, liens web.
- **Veille technique et technologique**
  Contexte : Répertorier articles techniques, blogs ou vidéos tech, résumés.
  Documents / Liens : PDF, texte, liens web.

Vous pouvez proposer un autre domaine, sous réserve de validation par votre hiérarchie.

Caractéristiques techniques

Cette application doit permettre la consultation, l'ajout, la modification et la suppression de ressources, qu'il s'agisse de documents ou de liens web, ainsi que le téléversement de fichiers pour les documents, sans recourir à une base de données.

Le stockage des métadonnées se fera dans un fichier texte **`data/ressources.txt`** au format :

```
clé:"valeur"
```

- Les valeurs sont encadrées par des guillemets et peuvent contenir plusieurs lignes.
- Une ligne vide sépare chaque document ou favori.
- Le caractère « : » est autorisé dans les valeurs grâce aux guillemets.
- Le caractère « " » (guillemet droit) **ne doit pas figurer** dans les valeurs, car il sert de délimiteur. Votre application doit le remplacer (par exemple par l'entité HTML **"**).

Voici un exemple de fichier **`data/ressources.txt`**, pour un document et un lien web :

```
id:"1"
type:"document"
titre:"Cours PHP"
mots-cles:"développement, web, php"
description:"Cours complet sur PHP
avec plusieurs chapitres et exemples pratiques."
fichier:"document_1.pdf"
date:"2025-11-14"

id:"2"
type:"favori"
titre:"10 règles d'or en matière de sécurité numérique"
mots-cles:"cybersécurité, numérique, mot de passe, mail, virus, malware "
description:"La réduction des risques induits par l'usage des technologies
numériques repose d'abord sur le respect de bonnes pratiques à adopter. À
elles seules, ces dix règles d'or vous protègeront de la majorité des
risques numériques qui pèsent sur vos usages personnels ou professionnels."
url:"https://cyber.gouv.fr/10-regles-dor-en-matiere-de-securite-
numerique"
date:"2025-08-07"
```

Vous disposez de fonctions PHP pour ajouter, lire, modifier et supprimer des ressources (CRUD).

Ces fonctions vous sont fournies sans documentation : c'est à vous de les comprendre (rôle, paramètres, valeur retournée, effets sur le fichier). Vous en rédigerez la documentation (voir Livrables).

Vous adopterez l'arborescence suivante :

- **index.php**
- **ajouter.php**
- **modifier.php**
- **supprimer.php**
- **includes**
  - **fonctions.php**
- **data**
  - **ressources.txt**
- **uploads**
  - **document_1.pdf**
  - **document_2.docx**
  - **document_3.jpg**
- **style**
  - **style.css**

Les documents et les favoris pourront être filtrés lors de leur affichage selon un ou plusieurs mots-clés. Pour ce faire, vous modifierez la fonction **`lire()`** en lui fournissant les mots-clés en argument.

Règles du filtre :

- Logique ET : une ressource n'est affichée que si *tous* les mots-clés saisis lui correspondent.
- La comparaison ne tient compte ni de la casse ni des accents (« Cybersécurité », « cybersecurite » et « CYBERSÉCURITÉ » sont équivalents).
- La correspondance est totale : un mot saisi doit être identique à l'un des mots-clés de la ressource (« php » ne correspond pas à « phpmyadmin »).
- Le filtre porte sur le champ mots-cles.
- Les mots-clés sont saisis séparés par des virgules.
- Sans mot-clé saisi, toutes les ressources sont affichées.
- Si aucune ressource ne correspond aux mots-clés, Le message « aucun résultat » est affiché par index.php.
- Le format de l'argument passé à lire() (chaîne, tableau…) est libre : c'est à vous de choisir et de justifier votre choix.
- Utilisez normaliser() pour la comparaison de la casse et des accents.

Pièges à anticiper

Dressez vous-même la liste des cas limites à tester : mots-clés avec espaces superflus avant/après ou multiples, virgules en double ou terminale, saisie uniquement d'espaces ou de virgules, mots-clés accentués/majuscules, mot partiel, doublons, ressource sans mot-clé. Consignez ces cas et leurs résultats dans les tests du README.

Objectifs

1. Comprendre et manipuler le tableau PHP représentant des ressources (documents ou favoris).
2. Intégrer les fonctions PHP fournies dans une application web.
3. Modifier une fonction PHP fournie
4. Créer des pages HTML / CSS pour :
    - Afficher la liste des ressources (documents ou favoris)
    - Ajouter un nouveau document ou un lien
    - Modifier un document ou un lien
    - Supprimer un document ou un lien après confirmation
5. Gérer les téléversements (uploads) de fichiers.
6. Tester et valider le fonctionnement complet de l'application.
7. Documenter votre code et versionner votre travail avec Git/GitHub.

Livrables

1. Pages PHP :
    - **`index.php`** : liste des ressources, avec filtre par mots-clés.
    - **`ajouter.php`** : formulaire d'ajout
    - **`modifier.php`** : formulaire d'édition
    - **`supprimer.php`** : confirmation de suppression
2. Dossier **`uploads/`** pour stocker les fichiers téléversés.
3. Fichier **`data/ressources.txt`** comportant les métadonnées des documents et des favoris.
4. Fichier **`includes/fonctions.php`** (fourni), avec modification de la fonction **`lire()`**.
5. Fichier **`style/style.css`**.
6. Fichier **README.md** contenant :
    a) La documentation des fonctions fournies (rôle, paramètres, valeur retournée) ;
    b) Vos choix ergonomiques et techniques (format de l'argument de lire(), choix de la bascule du formulaire…) ;
    c) La liste des tests réalisés et leurs résultats ;
    d) Si vous avez utilisé une IA, la mention de l'usage ; cette mention est obligatoire même sans usage : écrivez « aucun usage ».
    e) Le retour du camarade testeur : ce qu'il a trouvé bloquant, ce que vous avez corrigé ;
    f) Les difficultés rencontrées et les solutions.

Modalités de rendu

Organisez votre temps : le travail est évalué également sur la régularité de l'historique Git.

- **Date limite : 18/12/2026.**
- **Format :** dépôt GitHub (lien à transmettre dès la première séance de TP). Le dépôt doit contenir l'arborescence complète et un historique de *commits* réguliers.
- **Durée :** 15 heures de cours, complétées par du travail personnel.

Consignes

- Préparer l'environnement
  - Créer l'arborescence des dossiers du projet et les fichiers nécessaires.
  - Vérifier les permissions pour écrire dans **`data/`** et **`uploads/`**.
  - Initialiser le dépôt Git dès le début et faire un commit à chaque fonctionnalité terminée.
  - Valider vos pages avec le validateur W3C et corriger les erreurs.
- Préparer un jeu de données avant de programmer la page **`index.php`**
  - Saisissez le contenu du fichier **`data/ressources.txt`** : au moins 15 ressources, dont au moins 6 favoris et 6 documents, avec des mots-clés communs à plusieurs ressources pour pouvoir tester le filtre. Le jeu doit aussi comporter des ressources permettant de tester les pièges (accents, majuscules, mots proches comme php/phpmyadmin, espaces superflus).
  - Ajouter des documents dans le dossier **`uploads`**
- Créer la page **`index.php`**
  - Afficher tous les documents et favoris
  - Inclure des liens pour :
    - Ajouter une ressource
    - Voir chaque ressource : le fichier ou l'URL s'ouvre dans un nouvel onglet (avec rel="noopener noreferrer" pour les liens externes)
    - Modifier chaque ressource
    - Supprimer chaque ressource
  - Le type de chaque ressource (document ou favori) doit être visuellement identifiable
- Créer la page **`ajouter.php`**
  - Un seul formulaire permet d'ajouter un document ou un favori. Selon le type choisi, le champ pertinent (téléversement de fichier ou URL) est affiché et l'autre masqué, grâce à quelques lignes de JavaScript. Une seule fonctionnalité JavaScript est attendue : la bascule du formulaire ; aucun autre usage n'est requis.
  - Le titre est obligatoire. Pour un document, un fichier est obligatoire ; pour un favori, une URL est obligatoire.
  - Les fichiers téléversés sont limités à une liste d'extensions autorisées : pdf, jpg, jpeg, png, gif, mp3, mp4, txt, docx, odt. Un message d'erreur clair est affiché si l'extension est refusée.
  - Les champs vides et le caractère « " » sont gérés : message d'erreur explicite, sans perte des données déjà saisies
- Créer la page **`modifier.php`**
  - Formulaire prérempli avec les valeurs de la ressource sélectionnée.
  - Pour un document, le fichier peut être remplacé (mise à jour) ; s'il n'est pas remplacé, l'ancien fichier est conservé.
  - Le type d'une ressource ne peut pas être modifié : on ne peut pas passer d'un document à un favori ou inversement.
- Créer la page **`supprimer.php`**
  - Afficher le titre de la ressource à supprimer.
  - Demander confirmation avec boutons "Oui / Annuler".
  - Pour un document, le fichier correspondant est également supprimé du dossier uploads.
- Modifier la page **`index.php`**
  - Ajouter le filtre pour les mots-clés
  - Modifier la fonction **`lire()`**
  - Afficher tous les documents et favoris selon le contenu du filtre
  - Conserver les mots-clés saisis dans le champ après l'envoi, proposer un lien « Réinitialiser », et afficher un message si aucun résultat.
- Tests
  - Testez chaque fonctionnalité au fur et à mesure et consignez dans le README une liste des tests réalisés (action, résultat attendu, résultat obtenu). Un tableau de 8 à 10 lignes suffit.

Exigences minimales de sécurité et de robustesse

- Toute valeur issue du fichier ou d'un formulaire et affichée dans une page HTML est protégée avec htmlspecialchars().
- Les champs obligatoires sont vérifiés côté serveur avant tout enregistrement.
- Le caractère « " » est remplacé par une entité HTML.
- Les extensions des fichiers téléversés sont contrôlées (liste blanche).
- Les identifiants reçus dans l'URL (?id=...) sont vérifiés : si l'identifiant n'existe pas, un message d'erreur est affiché au lieu d'un plantage.
- Le fichier téléversé est obligatoirement renommé côté serveur (nom généré, jamais le nom d'origine fourni par l'utilisateur).

Critères d'évaluation

Critère	Détail	Points
Qualité HTML/CSS	HTML valide et sémantique, vérifié avec le validateur W3C (1,5) · CSS externe organisée (1) · ergonomie (1,5)	4
Affichage des ressources	Champs pertinents (1) · distinction document/favori (1) · liens « voir » fonctionnels (1)	3
Ajout	Formulaire unique avec bascule JS (1) · enregistrement correct (1) · contrôle des saisies et messages d'erreur (2)	4
Fichiers uploadés	Téléversement, contrôle des extensions et renommage (1,5) · remplacement à la modification et suppression à la suppression (1,5)	3
Modification	Formulaire prérempli (1) · mise à jour correcte des données (1)	2
Suppression	Page de confirmation avec titre affiché (1) · suppression effective après confirmation (1)	2
Filtre mots-clés	lire() respecte les règles et pièges (2) · intégration dans index.php (2)	4
Sécurité et robustesse	htmlspecialchars, validations serveur, vérification des identifiants, renommage	3
Code PHP	Noms de variables clairs, indentation, commentaires	2
Compréhension et documentation	README : documentation des fonctions fournies (1) · choix justifiés et difficultés rencontrées (1) · tests pertinents incluant les pièges et retour du camarade testeur (1)	3
Total		**30**

Historique Git

La progression de votre travail doit être visible dans l'historique (un commit à chaque fonctionnalité opérationnelle, avec un message explicite). Un historique irrégulier ou limité à quelques commits en fin de projet entraîne un retrait de points sur la note finale, pouvant atteindre 15 points sur 30 si le dépôt ne contient qu'un seul commit.

Le retrait s'applique après calcul des points des autres critères.

Qualité de l'historique	Retrait
Régulier (au moins un commit par fonctionnalité, répartis sur plusieurs séances, messages explicites)	0 point
Correct mais irrégulier (quelques gros commits ou messages peu explicites type « maj », « test »)	de 1 à 3 points
Insuffisant (moins de 5 commits ou presque tous le même jour)	de 4 à 8 points
Quasi inexistant (un seul commit, ou 2-3 commits regroupant tout le projet)	de 9 à 15 points

Conseils

- Bien comprendre le cahier des charges avant de commencer à coder
- Concevoir une interface ergonomique (facile à comprendre et à utiliser)
- Comprendre les fonctions fournies avant de les utiliser (arguments fournis, opérations réalisées)
- Comprendre comment parcourir un tableau en PHP
- Donner des noms clairs à vos variables et tableaux
- Tester votre code très régulièrement, ne pas tout coder d'un coup
- Tester chaque fonctionnalité séparément (imaginer des saisies qui pourraient faire bugger votre code PHP)
- Gérer les erreurs prévisibles
- Indenter et commenter votre code
- Faire tester votre application par un camarade pendant quelques minutes, sans explication : ce qui le bloque révèle les défauts d'ergonomie. Consignez son retour dans le README.
- Faire un commit après chaque fonctionnalité opérationnelle.
- Valider vos pages avec le validateur W3C et corriger les erreurs.

Consignes d'utilisation de l'IA

1. **L'IA est un outil d'apprentissage, pas un moyen de se soustraire à l'effort**
   Vous pouvez utiliser des assistants IA (ChatGPT, Copilot, etc.) pour comprendre un concept, obtenir des explications ou explorer des idées, mais copier-coller du code généré constitue une triche. Le TP doit refléter votre compréhension et votre travail personnel.
2. **Réfléchir avant de demander l'aide de l'IA**
   Avant de solliciter l'IA, essayez de résoudre le problème par vous-même :
   - Rédigez un pseudo-code ou un plan d'action.
   - Identifiez les différentes étapes et les tests à réaliser.
3. **Tester et vérifier**
   Le code généré par l'IA peut contenir des erreurs ou des failles :
   - Vérifiez le fonctionnement avec vos propres tests.
   - Analysez les performances et les limites de votre solution.
4. **L'intégrité académique prime**
   Toute soumission contenant du code copié directement d'une IA sera considérée comme un plagiat. Les TP seront évalués non seulement sur le résultat final, mais aussi sur la qualité de la démarche et de la compréhension. Vous devez être capable d'expliquer chaque ligne de votre code si je vous le demande.